GEP 프로젝트 고팀장 인수인계서

문서 코드: GEP-009
버전: 1.0
작성일: 2026.02.17
작성자: 노팀장
수신: 고팀장 (Claude Code)
승인: 조대표님

📋 프로젝트 개요

서비스 한 줄 요약

> 보험중개사 시험 합격을 위한 기출문제 반복 학습 플랫폼

협업 체계

역할	담당자	책임
의사결정	조대표님	모든 중요 결정 최종 승인
기획/맥락	노팀장	개발 지시, 검증, 맥락 유지
개발	고팀장	실제 개발, 결과 보고

보고 프로세스

```
고팀장 개발 완료
  ↓
노팀장 1차 검증
  ↓
조대표님 최종 승인
  ↓
확정
```

---

1. 즉시 착수 사항 (Week 0, 3일)

⚠️ 최우선 작업: 엑셀→노션 자동화

목적: 960문제(23~30회) 노션 DB 자동 이관

Step 1: 환경 세팅

```bash
```

pip install pandas notion-client openpyxl
```

**Step 2: 과목 매핑 테이블 확인**

엑셀 원본 → 서비스 표기 변환

|엑셀 원본 |서비스 표기 |소속|
|--------|---------|--|
|법령(보험업법)|보험업법 |1부|
|법령(상법) |상법 보험편 |1부|
|법령(위험관리)|위험관리론 |1부|
|회계 |세제 및 재무설계|1부|
|1부(자동차) |자동차보험 |2부|
|1부(특종) |특종보험 |2부|
|1부(보증) |보증보험 |2부|
|1부(연금) |개인연금·저축성 |2부|
|2부(화재) |화재보험 |3부|
|2부(해상) |해상보험 |3부|
|2부(항공) |항공·우주 |3부|
|2부(재보험) |재보험 |3부|

**Step 3: 스크립트 작성 (scripts/excel_to_notion.py)**

```python
import pandas as pd
from notion_client import Client
import os

# 설정
NOTION_TOKEN = os.getenv('NOTION_TOKEN')
NOTION_DB_ID = os.getenv('NOTION_DB_ID')
EXCEL_PATH = 'ExamBank_Master_Data_V4_5.xlsx'

# 과목 매핑
SUBJECT_MAP = {
    '보험업법': '보험업법',
    '상법': '상법 보험편',
    '위험관리': '위험관리론',
    '세제재무': '세제 및 재무설계',
    '자동차보험': '자동차보험',
    '특종보험': '특종보험',
    '보증보험': '보증보험',
    '연금저축': '개인연금·저축성',
    '화재보험': '화재보험',
    '해상보험': '해상보험',
    '항공우주': '항공·우주',
    '재보험': '재보험'
```
```

```
}

PART_MAP = {
    '법령': '1부', '회계': '1부',
    '1부': '2부', '2부': '3부'
}

def main():
    # 1. 엑셀 로드 (23~30회만)
    df = pd.read_excel(EXCEL_PATH, sheet_name='Sheet1')
    df = df[df['회차'] >= 23].reset_index(drop=True)
    print(f"✅ 로드 완료: {len(df)}문제")

    # 2. 노션 연결
    notion = Client(auth=NOTION_TOKEN)

    # 3. 삽입
    success = 0
    for _, row in df.iterrows():
        try:
            subject = SUBJECT_MAP.get(row['세부과목'], row['세부과목'])
            part = PART_MAP.get(row['과목'], row['과목'])

            notion.pages.create(
                parent={"database_id": NOTION_DB_ID},
                properties={
                    "문제ID": {
                        "title": [{"text": {"content": str(row['ID'])}}]
                    },
                    "회차": {"number": int(row['회차'])},
                    "문번": {"number": int(row['문제번호'])},
                    "과목": {"select": {"name": part}},
                    "세부과목": {"select": {"name": subject}},
                    "문제전체": {
                        "rich_text": [{"text": {"content": str(row['문제+보기'])}}]
                    },
                    "정답": {"number": int(row['정답'])},
                    "공개여부": {"checkbox": True}
                }
            )
            success += 1
            if success % 50 == 0:
                print(f"  진행중: {success}/{len(df)}")

        except Exception as e:
            print(f"❌ 오류 ({row['ID']}): {e}")

    print(f"\n✅ 완료: {success}/{len(df)}문제 이관")
```

```
if __name__ == '__main__':
    main()
```

Step 4: 노션→JSON 변환 (scripts/export_notion.py)

```python
import json
from notion_client import Client
import os

NOTION_TOKEN = os.getenv('NOTION_TOKEN')
NOTION_DB_ID = os.getenv('NOTION_DB_ID')
OUTPUT_PATH = 'public/data/exams.json'

def main():
    notion = Client(auth=NOTION_TOKEN)

    # 전체 문제 조회
    all_pages = []
    has_more = True
    start_cursor = None

    while has_more:
        response = notion.databases.query(
            database_id=NOTION_DB_ID,
            filter={"property": "공개여부", "checkbox": {"equals": True}},
            sorts=[
                {"property": "회차", "direction": "ascending"},
                {"property": "문번", "direction": "ascending"}
            ],
            start_cursor=start_cursor
        )
        all_pages.extend(response['results'])
        has_more = response['has_more']
        start_cursor = response.get('next_cursor')

    # JSON 변환
    exams = []
    for page in all_pages:
        props = page['properties']
        try:
            exams.append({
                'id': props['문제ID']['title'][0]['text']['content'],
                'round': props['회차']['number'],
                'number': props['문번']['number'],
                'part': props['과목']['select']['name'],
```

```
            'subject': props['세부과목']['select']['name'],
            'question': props['문제전체']['rich_text'][0]['text']['content'],
            'answer': props['정답']['number']
        })
    except Exception as e:
        print(f"⚠️ 파싱 오류: {e}")

# 저장
os.makedirs('public/data', exist_ok=True)
with open(OUTPUT_PATH, 'w', encoding='utf-8') as f:
    json.dump(exams, f, ensure_ascii=False, indent=2)

print(f"✅ JSON 저장: {len(exams)}문제 → {OUTPUT_PATH}")

if __name__ == '__main__':
    main()
```

Step 5: GitHub Actions (.github/workflows/sync-notion.yml)

```yaml
name: Sync Notion to JSON

on:
  schedule:
    - cron: '0 0 * * 0'  # 매주 일요일 0시
  workflow_dispatch:      # 수동 실행

jobs:
  sync:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install notion-client

      - name: Export Notion to JSON
        env:
          NOTION_TOKEN: ${{ secrets.NOTION_TOKEN }}
          NOTION_DB_ID: ${{ secrets.NOTION_DB_ID }}
        run: python scripts/export_notion.py
```

```
      - name: Commit and Push
        run: |
          git config user.name "GitHub Actions"
          git config user.email "actions@github.com"
          git add public/data/exams.json
          git diff --staged --quiet || git commit -m "chore: update exam data"
          git push
```

Week 0 완료 기준:

- [ ] 960문제 노션 이관 완료
- [ ] JSON 파일 생성 확인
- [ ] GitHub Actions 테스트 실행 성공
- [ ] 조대표님 데이터 검증 완료 (2시간)

---

2. Week 1: 핵심 기능 개발

프로젝트 초기 세팅

```bash
# 프로젝트 생성
npm create vite@latest gep-frontend -- --template react
cd gep-frontend

# 패키지 설치
npm install
npm install zustand
npm install -D tailwindcss postcss autoprefixer
npm install react-router-dom

# Tailwind 초기화
npx tailwindcss init -p
```

tailwind.config.js:

```javascript
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: { extend: {} },
  plugins: [],
}
```

핵심 개발 순서

1순위: QuestionView + AnswerButtons

```jsx
// src/components/QuestionView.jsx
export default function QuestionView({
  questionText,
  questionNumber,
  totalQuestions,
  selectedAnswer,
  showResult,
  correctAnswer,
  onAnswerSelect
}) {
  return (
    <div className="flex flex-col h-screen">
      {/* 문제 텍스트 영역 (스크롤) */}
      <div className="flex-1 overflow-y-auto p-4 pb-32">
        <p className="text-gray-500 text-sm mb-2">
          {questionNumber} / {totalQuestions}
        </p>
        <div
          className="text-base leading-relaxed whitespace-pre-wrap"
          dangerouslySetInnerHTML={{ __html: questionText }}
        />
      </div>

      {/* 답안 버튼 (하단 고정) */}
      <AnswerButtons
        selected={selectedAnswer}
        correct={correctAnswer}
        showResult={showResult}
        onSelect={onAnswerSelect}
      />
    </div>
  );
}
```

```jsx
// src/components/AnswerButtons.jsx
export default function AnswerButtons({
  selected,
  correct,
  showResult,
  onSelect
}) {
  const getButtonStyle = (num) => {
```

```
    if (!showResult) {
      return selected === num
        ? 'bg-blue-500 text-white'
        : 'bg-gray-100 text-gray-800';
    }
    if (num === correct) return 'bg-green-500 text-white';
    if (num === selected && num !== correct) return 'bg-red-400 text-white';
    return 'bg-gray-100 text-gray-400';
  };

  return (
    <div
      className="fixed bottom-0 left-0 right-0 bg-white border-t p-4"
      style={{ paddingBottom: 'env(safe-area-inset-bottom)' }}
    >
      <div className="grid grid-cols-2 gap-3">
        {[1, 2, 3, 4].map(num => (
          <button
            key={num}
            className={`
              h-16 rounded-xl text-xl font-bold
              transition-all duration-150
              ${getButtonStyle(num)}
            `}
            onClick={() => !showResult && onSelect(num)}
          >
            {num}
          </button>
        ))}
      </div>
    </div>
  );
}
```

2순위: Zustand Store

```javascript
// src/stores/examStore.js
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

const useExamStore = create(
  persist(
    (set, get) => ({
      // 상태
      allQuestions: [],
      currentRound: 30,
```

```
    currentSubject: null,
    currentIndex: 0,
    userAnswers: {},
    showResult: false,

    // 액션
    setAllQuestions: (q) => set({ allQuestions: q }),
    setRound: (round) => set({ currentRound: round, currentIndex: 0, currentSubject: null }),
    setSubject: (subject) => set({ currentSubject: subject, currentIndex: 0 }),
    setCurrentIndex: (i) => set({ currentIndex: i }),
    submitAnswer: (questionId, answer) => set((state) => ({
      userAnswers: { ...state.userAnswers, [questionId]: answer },
      showResult: true
    })),
    nextQuestion: () => set((state) => ({
      currentIndex: state.currentIndex + 1,
      showResult: false
    })),

    // 현재 문제 목록 계산
    getCurrentQuestions: () => {
      const { allQuestions, currentRound, currentSubject } = get();
      let filtered = allQuestions.filter(q => q.round === currentRound);
      if (currentSubject) {
        filtered = filtered.filter(q => q.subject === currentSubject);
      }
      return filtered;
    },
  }),
  { name: 'gep_store' }
 )
);

export default useExamStore;
```

3순위: localStorage 유틸

```javascript
// src/utils/storage.js

export const storage = {
 // 진도 저장
 saveProgress(round, subject, index) {
   const key = subject
     ? `gep_progress_${round}_${subject}`
     : `gep_progress_${round}`;
   localStorage.setItem(key, String(index));
```

```
  },

  // 진도 불러오기
  loadProgress(round, subject) {
    const key = subject
      ? `gep_progress_${round}_${subject}`
      : `gep_progress_${round}`;
    return parseInt(localStorage.getItem(key) || '0');
  },

  // 이력 저장
  saveHistory(entry) {
    const history = this.loadHistory();
    history.push(entry);
    localStorage.setItem('gep_history', JSON.stringify(history));
  },

  // 이력 불러오기
  loadHistory() {
    return JSON.parse(localStorage.getItem('gep_history') || '[]');
  },

  // 설정 저장
  saveSettings(settings) {
    localStorage.setItem('gep_settings', JSON.stringify(settings));
  },

  // 설정 불러오기
  loadSettings() {
    return JSON.parse(
      localStorage.getItem('gep_settings') ||
      '{"showAnswerImmediately":true,"fontSize":"medium"}'
    );
  },

  // 최근 학습 저장
  saveLastStudy(round, subject, index) {
    localStorage.setItem('gep_last', JSON.stringify({ round, subject, index }));
  },

  // 최근 학습 불러오기
  loadLastStudy() {
    return JSON.parse(localStorage.getItem('gep_last') || 'null');
  }
};
```

Week 1 완료 기준:

- [ ] 1문제 풀고 정답 확인 가능
- [ ] localStorage 저장 확인

---

3. Week 2: 데이터 연동 & 네비게이션

JSON 로드

```javascript
// src/utils/loadExams.js
export async function loadExams() {
  try {
    const res = await fetch('/data/exams.json');
    if (!res.ok) throw new Error('로드 실패');
    return await res.json();
  } catch (e) {
    console.error(e);
    return null;
  }
}
```

세부과목 드롭다운 (모달 없음)

```jsx
// src/components/SubjectDropdown.jsx
const SUBJECTS = [
  { id: '보험업법', part: '1부' },
  { id: '상법 보험편', part: '1부' },
  { id: '위험관리론', part: '1부' },
  { id: '세제 및 재무설계', part: '1부' },
  { id: '자동차보험', part: '2부' },
  { id: '특종보험', part: '2부' },
  { id: '보증보험', part: '2부' },
  { id: '개인연금·저축성', part: '2부' },
  { id: '화재보험', part: '3부' },
  { id: '해상보험', part: '3부' },
  { id: '항공·우주', part: '3부' },
  { id: '재보험', part: '3부' },
];

export default function SubjectDropdown({ current, onChange }) {
  return (
    <select
      value={current || '전체'}
      onChange={(e) => onChange(e.target.value === '전체' ? null : e.target.value)}
```

```
      className="text-sm border rounded px-2 py-1"
    >
      <option value="전체">전체</option>
      {SUBJECTS.map(s => (
        <option key={s.id} value={s.id}>{s.id}</option>
      ))}
    </select>
  );
}
```

React Router 설정

```jsx
// src/App.jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import Question from './pages/Question';
import Settings from './pages/Settings';

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/question" element={<Question />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Week 2 완료 기준:

- [ ] 960문제 전체 풀이 가능
- [ ] 세부과목 드롭다운 전환 정상
- [ ] 이어하기 기능 정상

---

4. Week 3: 완성 & 배포

PWA 설정

```javascript
// vite.config.js
import { defineConfig } from 'vite';
```

```
import react from '@vitejs/plugin-react';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      manifest: {
        name: 'GEP 보험중개사',
        short_name: 'GEP',
        description: '보험중개사 기출문제 학습',
        theme_color: '#3B82F6',
        background_color: '#ffffff',
        display: 'standalone',
        icons: [
          { src: '/icons/icon-192.png', sizes: '192x192', type: 'image/png' },
          { src: '/icons/icon-512.png', sizes: '512x512', type: 'image/png' }
        ]
      }
    })
  ]
});
```

에러 처리

```jsx
// src/pages/Question.jsx 에러 처리
const [error, setError] = useState(null);

if (error) {
  return (
    <div className="flex flex-col items-center justify-center h-screen p-8">
      <p className="text-gray-500 text-center mb-4">
        데이터를 불러올 수 없습니다.<br/>
        새로고침 후 다시 시도해주세요.
      </p>
      <button
        className="bg-blue-500 text-white px-6 py-3 rounded-xl"
        onClick={() => window.location.reload()}
      >
        새로고침
      </button>
    </div>
  );
}
```

Vercel 배포

```bash
# Vercel CLI 설치
npm install -g vercel

# 배포
vercel --prod

# 환경변수 설정 (Vercel 대시보드)
# NOTION_TOKEN = [노션 API 키]
# NOTION_DB_ID = [DB ID]
```

Week 3 완료 기준:

- [ ] PWA 홈 화면 추가 가능
- [ ] Vercel 배포 완료
- [ ] 전체 기능 테스트 완료
- [ ] 베타 사용자 10명 테스트

---

5. 보고 규칙

노팀장에게 보고 시점

1. **Week 0 완료:** 노션 DB 구축 완료
1. **Week 1 완료:** 문제 풀이 + 저장 가능
1. **Week 2 완료:** 전체 960문제 풀이 가능
1. **이슈 발생:** 즉시 보고

보고 형식

```
[완료/이슈/질문]
- 작업 내용:
- 결과:
- 다음 작업:
- 질문/이슈:
```

의사결정 필요 시

- 노팀장 → 조대표님 보고
- 고팀장 임의 결정 금지

---

6. 환경변수 목록

변수명	설명	발급처
NOTION_TOKEN	노션 API 키	노션 통합 설정
NOTION_DB_ID	노션 DB ID	노션 DB URL
VITE_APP_VERSION	앱 버전	직접 설정

---

7. 주요 참조 문서

문서	내용
GEP-001 서비스기획 2.0	전체 서비스 방향
GEP-005 백엔드기획 3.0	데이터 구조, Phase별 전략
GEP-007 프론트엔드기획 2.0	UI/UX, 컴포넌트 명세
GEP-008 통합검증 최종	확정 결정사항 전체

📌 최종 확인

작성자: 노팀장
수신: 고팀장
승인: 조대표님
날짜: 2026.02.17

개발 착수 조건:

- [x] 조대표님 승인 완료
- [x] 기획안 3개 완성
- [x] 인수인계서 완성
- [ ] **고팀장 Week 0 착수**

노팀장 드림